

CE49X: Introduction to Computational Thinking and Data Science for Civil Engineers

Week 3: Python Modules, Strings, and Data Science Tools

Dr. Eyuphan Koc

Department of Civil Engineering
Bogazici University

Fall 2025

Based on "A Whirlwind Tour of Python" by Jake VanderPlas
Chapters 13-15: Modules & Packages, Strings & Regular Expressions, Data Science Tools Preview
<https://github.com/jakevdp/WhirlwindTourOfPython>

- 1 Modules and Packages
- 2 String Manipulation and Regular Expressions
- 3 Preview of Data Science Tools
- 4 Practical Engineering Applications
- 5 Week 3 Summary and Next Steps

Python's "Batteries Included" Philosophy

What Makes Python Powerful?

- **Built-in modules:** Ready-to-use functionality
- **Third-party packages:** Extensive ecosystem (100,000+ packages)
- **Easy installation:** Package managers like pip and conda
- **Modular design:** Organize code into reusable components

Civil Engineering Applications

- Access FEM analysis libraries (OpenSeesPy, FEniCS)
- Use structural design modules (PyNite, StructPy)
- Integrate with BIM software (IfcOpenShell)
- Build custom analysis tools for your specific needs

Today's Goal

Master Python's module system to leverage existing tools and build your own!

Understanding Modules vs Packages

Module

- Single Python file (.py)
- Contains functions, classes, variables
- Example: beam_analysis.py
- Imported with `import beam_analysis`

```
1 # beam_analysis.py
2 def calculate_moment(load, span):
3     return load * span**2 / 8
```

Package

- Directory containing multiple modules
- Has `__init__.py` file
- Example: structural/ package
- Contains many sub-modules

```
1 # structural/
2 # __init__.py
3 # beams.py
4 # columns.py
5 # loads.py
```

Think of it this way

Module = One tool (hammer) | Package = Toolbox (complete set)

Import Methods: Best Practices

1. Explicit Module Import (Recommended)

Preserves namespace clarity, easy to trace function origins, prevents naming conflicts.

```
1 import math
2 import statistics # <-- Try importing this!
3
4 # Clear where functions come from
5 beam_angle = math.cos(math.pi/4) # From math module
6 load_avg = statistics.mean([10, 15, 20, 25]) # From statistics
7
8 print(f"Angle: {beam_angle:.3f}")
9 print(f"Average load: {load_avg} kN")
```

[QUICK] Try it yourself (1 minute)

Import the datetime module and print today's date! Hint: Use `datetime.date.today()`

Import Aliases for Convenience

2. Module Import with Alias

Shorter names, standard scientific conventions, maintains namespace separation.

```
1 import numpy as np # Standard alias # --> Predict: What will np stand for?
2 import matplotlib.pyplot as plt # For plotting
3 import pandas as pd # For data analysis
4
5 # Standard scientific Python conventions
6 loads = np.array([120, 145, 98, 167, 134]) # kN
7 stresses = loads / 25 # MPa (assuming area = 25 cm^2)
8
9 print(f"Max stress: {np.max(stresses):.1f} MPa")
10 print(f"Mean stress: {np.mean(stresses):.1f} MPa")
```

Community Conventions

- np = NumPy (always)
- pd = Pandas (always)
- plt = Matplotlib.pyplot (always)
- Following these makes your code instantly recognizable!

Selective Imports

3. Import Specific Functions

Import only what you need, cleaner code, good for frequently used functions.

```
1 from math import cos, sin, pi, sqrt
2 from statistics import mean, stdev
3
4 # Structural vibration calculation # <-- Students: Can you spot the issue?
5 frequency = 2.5 # Hz
6 period = 1 / frequency
7 angle = 2 * pi * frequency * 0.1 # at t=0.1s
8
9 # Direct use without module prefix
10 amplitude = 10 * cos(angle) # mm
11 velocity = -10 * 2 * pi * frequency * sin(angle) # mm/s
12
13 print(f"Displacement: {amplitude:.2f} mm")
14 print(f"Velocity: {velocity:.2f} mm/s")
```

[DEBUG] Common Pitfall

What happens if you do: `from numpy import *` and then `sum([1, 2, 3])`? The NumPy `sum` overwrites Python's built-in `sum`!

[LIVE] Coding Challenge 2: Module Organization

Your Task (4 minutes)

Create a structural analysis module organization:

- Define functions for beam moment, shear, and deflection
- Import and use them correctly
- Calculate values for a 6m beam with 10 kN/m load

Starter Code

```
1 # Define your analysis functions
2 def calculate_moment(w, L):
3     # YOUR CODE HERE: return max moment for simply supported beam
4     pass
5
6 def calculate_shear(w, L):
7     # YOUR CODE HERE: return max shear
8     pass
9
10 def calculate_deflection(w, L, E=200000, I=8333):
11     # YOUR CODE HERE: return max deflection
12     pass
13
14 # Test with: w=10 kN/m, L=6m
```


Import Wildcards: When to Avoid

4. Wildcard Imports (Generally Avoid)

`from module import *` imports everything, can overwrite functions, makes debugging harder.

The Problem - A Real Debugging Nightmare

- Python's `sum([1, 2, 3])` returns 6
- NumPy's `sum([1, 2, 3])` returns `numpy.int64(6)`
- Different behavior can break your code silently!

Best Practice

- **Good:** `import numpy as np` or `from math import sin, cos`
- **Bad:** `from numpy import *`
- **Exception:** Interactive exploration in Jupyter (but fix before production)

Essential Standard Library Modules for Engineers

Mathematical & Scientific

- `math`: Trigonometry, logarithms
- `statistics`: Mean, stdev, regression
- `random`: Monte Carlo simulations
- `cmath`: Complex numbers (signals)

Data Processing

- `csv`: Read sensor data files
- `json`: API data exchange
- `pickle`: Save Python objects

System & File Operations

- `os`: File system navigation
- `pathlib`: Modern path handling
- `datetime`: Time series data

Advanced Tools

- `itertools`: Combinations, permutations
- `functools`: Caching, decorators
- `urllib`: Download data from web

[PRACTICE] Try This Now (2 minutes)

Import `random` and generate 5 random concrete strengths between 20-40 MPa

[TOGETHER] Installing Third-Party Packages

Package Installation Methods

- **pip:** `pip install numpy scipy matplotlib`
- **conda:** `conda install numpy scipy matplotlib`
- **Virtual environments:** Isolated project dependencies

```
1 # Check installed packages
2 import sys
3 !pip list | grep numpy # In Jupyter/Colab
4 # Install specific version: pip install numpy==1.21.0
5 # Install from requirements: pip install -r requirements.txt
```

Engineering Package Examples

- `pip install openseespy` – Structural analysis
- `pip install pynite` – Frame analysis
- `pip install ifcopenshell` – BIM/IFC files

Professional Tip

Create a `requirements.txt` file for each project to track dependencies!

Why String Processing Matters in Engineering

Common Engineering String Tasks

- **Data Import:** Reading CSV files, parsing measurement data
- **Report Generation:** Creating formatted output documents
- **File Processing:** Handling CAD files, analysis output files
- **Data Validation:** Checking input formats, units, ranges
- **Database Operations:** SQL queries, data cleaning

Real-World Examples

- Parsing bridge inspection reports from text files
- Extracting coordinates from survey data
- Formatting structural analysis results for presentations
- Processing weather station data files
- Converting between different data formats

Python String Fundamentals

String Definition Options

- **Single quotes:** 'Steel Grade 250'
- **Double quotes:** "Concrete fc'=30" (use when string contains ')
- **Triple quotes:** Multi-line strings, docstrings
- **Raw strings:** r"C:\Bridge\Data\sensors.csv" (no escapes)

```
1 # Engineering data with different string types
2 material = 'reinforced concrete' # Simple string
3 spec = "fc'=30 MPa" # Contains apostrophe # <-- Why double quotes?
4 report = """Structural Analysis Report
5 Date: 2025-01-15
6 Project: Highway Bridge"""
7
8 # File paths - always use raw strings on Windows!
9 data_file = r"C:\Projects\Bridge_2025\load_data.csv" # Correct
10 # bad_path = "C:\Projects\Bridge_2025\load_data.csv" # Will fail!
```

[DEBUG] Common Error

What's wrong with: `path = "C:\newfolder\test.txt"`? The `\n` becomes newline! Use raw strings: `r"C:\newfolder\test.txt"`

Essential String Methods for Engineering Data

```
1 # Processing messy engineering data # --> Predict the outputs!
2 material = "  REINFORCED concrete  "
3 clean = material.strip().title()
4 print(clean)  # ?
5
6 # Parsing member IDs
7 member_id = "BEAM-B01-LEVEL3"
8 parts = member_id.split('-') # ['BEAM', 'B01', 'LEVEL3']
9 beam_type = parts[0].lower() # 'beam'
10 beam_num = parts[1] # 'B01'
11
12 # Replacing units
13 measurement = "Load: 1500 kips"
14 metric = measurement.replace('kips', 'kN')
15 value = float(measurement.split()[1]) * 4.448 # Convert to kN
16 print(f"Metric: {value:.1f} kN")
```

Key String Methods

.strip() removes whitespace | .split() creates list | .join() combines list | .replace() substitutes text

[PRACTICE] String Cleaning Challenge

Your Task (3 minutes)

Clean and standardize this sensor data:

Starter Code

```
1 # Messy sensor readings
2 data = [
3     "  Sensor_01: 125.3 mPa  ",
4     "sensor_02:98.7mpa",
5     "SENSOR_03 : 145.2 MPA",
6     "  sensor-04: 112.8 Mpa"
7 ]
8
9 # YOUR TASK: Clean and standardize to format "S01: 125.3 MPa"
10 # Hints:
11 # 1. Extract sensor number
12 # 2. Extract value
13 # 3. Standardize units to "MPa"
14 # 4. Format as "S##: value MPa"
15
16 for reading in data:
17     # YOUR CODE HERE
18     pass
```

Modern String Formatting: f-strings

Why f-strings? (Python 3.6+)

Readable, fast, supports expressions, type formatting

```
1 # Engineering calculations with formatted output
2 beam_id = "B-001"
3 moment = 245.678 # kN*m
4 capacity = 300 # kN*m
5 utilization = moment / capacity * 100
6
7 # f-string with expressions # <-- Try changing precision!
8 report = f"""Beam Analysis Report
9 ID: {beam_id}
10 Moment: {moment:.1f} kN*m
11 Capacity: {capacity} kN*m
12 Utilization: {utilization:.1f}%
13 Status: {'OK' if utilization < 90 else 'CHECK REQUIRED'}"""
14
15 print(report)
16
17 # Advanced formatting
18 pi_value = 3.14159265
19 print(f"Pi to 3 decimals: {pi_value:.3f}")
20 print(f"Percentage: {0.856:.1%}") # Automatic % conversion!
21 print(f"Scientific: {1234567:.2e}")
```


What are Regular Expressions?

Pattern matching for complex text processing: find patterns, validate formats, extract data.

Engineering Applications

- Extract dimensions: "200x400x6000mm"
- Validate formats: coordinates, member IDs, loads
- Parse log files and reports
- Clean measurement data

Learning Tip

Start with simple patterns, build complexity gradually.

Pattern Matching Power

Extract complex patterns from text: measurements, IDs, coordinates

```
1 import re
2
3 # Engineering report with mixed data
4 report = """Bridge inspection 2025-01-15:
5 Beam B-001: 250x400mm, stress 145.3 MPa
6 Column C-42A: 600x600mm, load 1250.5 kN
7 Coordinates: (42.3567, -71.0589)
8 Next inspection: 2025-07-15"""
9
10 # Extract all measurements with units # --> Try these patterns!
11 nums_with_units = re.findall(r'\d+\.\d*\s*(?:mm|MPa|kN)', report)
12 print("Measurements:", nums_with_units)
13
14 # Extract member IDs (letter-numbers-optional letter)
15 member_ids = re.findall(r'[A-Z]-\d+[A-Z]?', report)
16 print("Members:", member_ids)
17
18 # Extract dates (YYYY-MM-DD)
19 dates = re.findall(r'\d{4}-\d{2}-\d{2}', report)
20 print("Dates:", dates)
```

[DEBUG] Together: Fix the Regex Patterns

Collaborative Debugging (4 minutes)

These patterns have bugs. Let's fix them together!

```
1 import re
2
3 # Bug #1: Coordinate extraction
4 coords = "Location: (42.3567, -71.0589) and (40.7128, -74.006)"
5 pattern1 = r'\d+\.\d+, \d+\.\d+' # What's missing? # <-- Students: Fix this!
6
7 # Bug #2: Dimension extraction
8 dims = "Steel section: 200x300x6000mm, Concrete: 400x800mm"
9 pattern2 = r'\d+x\d+' # Doesn't get all dimensions
10
11 # Bug #3: Load value extraction
12 loads = "Dead: 125.5kN, Live: 85 kN, Wind: 42.75kN"
13 pattern3 = r'\d+kN' # Missing something?
14
15 # Test and fix each pattern
16 print("Coords found:", re.findall(pattern1, coords))
17 print("Dimensions found:", re.findall(pattern2, dims))
18 print("Loads found:", re.findall(pattern3, loads))
```

Discuss: What patterns did you find? How would you fix them?

[COMPETITION] Regex Challenge: Data Extraction Race

Mini-Competition (5 minutes)

Extract all required data from this construction log. Most complete & efficient wins!

```
1 log = ""2025-01-15 08:30:45 - Pour started: Section A1-B2
2 Concrete: fc'=35MPa, Slump: 150mm, Temp: 22.5C
3 Volume: 45.5m3, Truck#: CT-2847, Driver: ID#8934
4 2025-01-15 11:45:20 - Pour complete: Section A1-B2
5 Total time: 3.25hrs, Weather: Clear, 18.5C
6 Next pour: Section B2-C3, Date: 2025-01-16""
7
8 # YOUR CHALLENGE: Extract:
9 # 1. All timestamps (YYYY-MM-DD HH:MM:SS)
10 # 2. All section IDs (A1-B2 format)
11 # 3. All measurements with units
12 # 4. All ID numbers (ID#XXXX or CT-XXXX)
13
14 # YOUR CODE HERE - Aim for elegance!
15
16
17 # Scoring: Correctness (50%), Code efficiency (30%), Readability (20%)
```

The Scientific Python Ecosystem

OpenSeesPy	FEniCS	PyNite
Pandas	Scikit-learn	SymPy Jupyter
NumPy	SciPy	Matplotlib
Python Core Language		

Build on Giants' Shoulders

Don't reinvent the wheel - use tested, optimized libraries!

Key Features

N-dimensional arrays, vectorized operations, linear algebra, 10-100x faster than Python lists.

Engineering Applications

- Store sensor measurement data
- Represent structural matrices
- Perform finite element calculations
- Process time-series monitoring data

[LIVE] NumPy: Foundation of Scientific Computing

Why NumPy?

10-100x faster than Python lists, foundation for all scientific libraries

```
1 import numpy as np
2
3 # Structural loads analysis # --> Predict: What's the speedup?
4 loads_list = [120, 145, 98, 167, 134] # Python list
5 loads_array = np.array(loads_list)    # NumPy array
6
7 # Compare operations
8 # Python way (slow)
9 factored_list = [x * 1.5 for x in loads_list]
10
11 # NumPy way (fast) - vectorized!
12 factored_array = loads_array * 1.5 # All at once!
13
14 print(f"Max load: {np.max(loads_array)} kN")
15 print(f"Mean: {np.mean(loads_array):.1f} kN")
16 print(f"Std Dev: {np.std(loads_array):.1f} kN")
17
18 # Matrix operations for structural analysis
19 K = np.array([[1000, -500], [-500, 1000]]) # Stiffness matrix
20 F = np.array([100, 50]) # Force vector
21 u = np.linalg.solve(K, F) # Solve  $K \cdot u = F$ 
22 print(f"Displacements: {u}")
```

[PRACTICE] NumPy Challenge: Beam Analysis

Your Task (4 minutes)

Use NumPy to analyze a simply supported beam with multiple point loads

Starter Code

```
1 import numpy as np
2
3 # Beam data
4 L = 10 # meters
5 loads = np.array([50, 30, 40, 20]) # kN
6 positions = np.array([2, 4, 6, 8]) # meters from left support
7
8 # YOUR TASK:
9 # 1. Calculate reactions at supports (RA and RB)
10 # 2. Calculate moment at each load position
11 # 3. Find maximum moment and its location
12
13 # Hint: For simply supported beam:
14 #  $RA = \frac{\sum(P * (L - x))}{L}$ 
15 #  $M(x) = RA * x - \sum(P)$  for loads before x
16
17 # YOUR CODE HERE
```


Key Features

DataFrame structures, data cleaning, import/export (CSV, Excel), grouping, time series.

Engineering Applications

- Import sensor data and test results
- Analyze time series monitoring
- Create statistical summaries
- Generate analysis reports

Why Pandas?

Excel-like functionality with Python power and reproducibility.

[TOGETHER] Pandas: Data Analysis Made Easy

```
1 import pandas as pd
2 import numpy as np
3
4 # Create concrete test data # <-- Students: Follow along!
5 concrete_data = {
6     'sample_id': ['C001', 'C002', 'C003', 'C004', 'C005'],
7     'age_days': [7, 7, 28, 28, 28],
8     'strength_mpa': [18.5, 19.2, 28.3, 31.5, 29.8],
9     'mix_type': ['A', 'A', 'B', 'B', 'B']
10 }
11
12 df = pd.DataFrame(concrete_data)
13 print(df)
14
15 # Analysis operations
16 print(f"\nMean 28-day strength: {df[df['age_days']==28]['strength_mpa'].mean():.1f} MPa")
17
18 # Group by mix type
19 stats = df.groupby('mix_type')['strength_mpa'].agg(['mean', 'std', 'count'])
20 print("\nStatistics by mix:")
21 print(stats)
22
23 # Add compliance check
24 df['compliant'] = df['strength_mpa'] >= 25
25 print(f"\nCompliance rate: {df['compliant'].mean():.1%}")
```

Key Features

Publication-quality plots, wide variety of chart types, complete customization, multiple output formats.

Engineering Visualizations

- Load-displacement curves
- Time series sensor data
- Stress distribution plots
- Material property distributions
- Project progress charts

[LIVE] Matplotlib: Publication-Quality Plots

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Generate structural response data # --> Try different parameters!
5 time = np.linspace(0, 10, 100) # seconds
6 frequency = 2.0 # Hz
7 damping = 0.1
8 amplitude = 10 * np.exp(-damping * time) * np.sin(2 * np.pi * frequency * time)
9
10 # Create professional plot
11 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
12
13 # Time series
14 ax1.plot(time, amplitude, 'b-', linewidth=2)
15 ax1.set_xlabel('Time (s)')
16 ax1.set_ylabel('Displacement (mm)')
17 ax1.set_title('Damped Vibration Response')
18 ax1.grid(True, alpha=0.3)
19 ax1.axhline(y=0, color='k', linestyle='-', linewidth=0.5)
20
21 # Histogram of peaks
22 peaks = amplitude[amplitude > 0]
23 ax2.hist(peaks, bins=20, edgecolor='black', alpha=0.7)
24 ax2.set_xlabel('Amplitude (mm)')
25 ax2.set_ylabel('Frequency')
26 ax2.set_title('Distribution of Positive Peaks')
27
28 plt.tight_layout()
29 plt.show()
```

[QUICK] SciPy: Advanced Scientific Computing

Key Capabilities

Optimization, curve fitting, interpolation, integration, signal processing

```
1 from scipy import optimize, interpolate
2 import numpy as np
3
4 # Curve fitting example - concrete strength vs time
5 days = np.array([3, 7, 14, 28, 56])
6 strength = np.array([12, 20, 26, 30, 32]) # MPa
7
8 # Fit exponential model:  $f(t) = a * (1 - \exp(-b*t))$ 
9 def model(t, a, b):
10     return a * (1 - np.exp(-b * t))
11
12 params, _ = optimize.curve_fit(model, days, strength)
13 print(f"Model:  $f(t) = \{params[0]:.1f\} * (1 - \exp(-\{params[1]:.3f\}*t))$ ")
14
15 # Interpolation for intermediate values
16 interp_func = interpolate.interp1d(days, strength, kind='cubic')
17 day_21_strength = interp_func(21)
18 print(f"Predicted 21-day strength:  $\{day\_21\_strength:.1f\}$  MPa")
```

Pair Exercise (2 minutes)

With your neighbor: What other SciPy functions would be useful for structural analysis?

[EXPLORE] Specialized Engineering Packages

Structural Analysis

- **OpenSeesPy**: Earthquake engineering
- **PyNite**: 3D frame analysis
- **anaStruct**: 2D frame analysis
- **FEniCS**: Finite element modeling

Data & Visualization

- **Plotly**: Interactive plots
- **Seaborn**: Statistical graphics
- **Bokeh**: Web visualizations

BIM & CAD

- **IfcOpenShell**: IFC file handling
- **PythonOCC**: CAD kernel
- **FreeCAD API**: CAD automation

Utilities

- **Pint**: Unit conversions
- **SymPy**: Symbolic math
- **Uncertainties**: Error propagation

[DISCUSS] Class Poll

Which package sounds most useful for your projects? Why?

[TOGETHER] Getting Started with Data Science

Installation Options

- **Anaconda:** Complete scientific distribution (recommended)
- **Miniconda:** Minimal conda installer
- **pip + venv:** Traditional Python approach

```
1 # Check your setup
2 import sys
3 print(f"Python: {sys.version}")
4 # Check essential packages
5 try:
6     import numpy as np
7     import pandas as pd
8     import matplotlib
9     print("Essential packages installed!")
10    print(f"NumPy version: {np.__version__}")
11 except ImportError as e:
12    print(f"Missing package: {e}")
```

Professional Workflow

1. Create virtual environment | 2. Install packages | 3. Save requirements.txt | 4. Version control

Real Engineering Workflow: From Data to Decision

Complete Data Science Pipeline

- 1 **Import:** Multiple data sources (CSV, Excel, databases)
- 2 **Clean:** Handle missing values, outliers, units
- 3 **Transform:** Calculate derived properties
- 4 **Analyze:** Statistical analysis, pattern detection
- 5 **Visualize:** Create publication-quality figures
- 6 **Report:** Generate automated reports

Today's Case Study

Bridge monitoring data: 1000+ sensors, 6 months of data, multiple formats

- Challenge: Identify anomalies and predict maintenance needs
- Tools: Pandas for processing, NumPy for calculations, Matplotlib for visualization

Industry Reality

80% of data science is data cleaning - let's automate it!

[COMPETITION] Data Pipeline Challenge

Team Competition (7 minutes)

Build the most efficient concrete quality control pipeline!

```
1 import pandas as pd
2 import numpy as np
3
4 # Sample data (you'll get more)
5 data = {
6     'batch_id': ['B001', 'B002', 'B003', 'B004', 'B005'],
7     'strength_7d': [18.5, 'N/A', 21.3, 19.8, 22.1],
8     'strength_28d': [28.3, 31.5, '29.8', 27.2, 33.4],
9     'cement_kg': [320, 350, 340, 330, 360],
10    'water_cement': [0.45, 0.42, 0.43, 0.44, 0.40]
11 }
12
13 # YOUR CHALLENGE:
14 # 1. Clean the data (handle 'N/A', convert types)
15 # 2. Calculate strength gain ratio (28d/7d)
16 # 3. Flag non-compliant batches (28d < 30 MPa)
17 # 4. Find correlation between w/c ratio and strength
18 # 5. Create summary statistics by cement content group
19
20 # YOUR CODE HERE - Most complete & elegant wins!
```

[LIVE] Complete Analysis Pipeline

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Real-world pipeline # --> Follow along and modify!
6 def analyze_concrete_data(filename):
7     # Import
8     df = pd.read_csv(filename)
9
10    # Clean
11    df['strength'] = pd.to_numeric(df['strength'], errors='coerce')
12    df = df.dropna()
13
14    # Transform
15    df['strength_ratio'] = df['strength'] / df['target_strength']
16    df['compliance'] = df['strength_ratio'] >= 1.0
17
18    # Analyze
19    summary = {
20        'total_samples': len(df),
21        'mean_strength': df['strength'].mean(),
22        'std_strength': df['strength'].std(),
23        'compliance_rate': df['compliance'].mean(),
24        'critical_samples': df[~df['compliance']]['sample_id'].tolist()
25    }
26
27    return df, summary
28
29 # Test the pipeline
30 # df, results = analyze_concrete_data('test_data.csv')
```

Modules & Packages

- Import strategies and best practices
- Standard library modules
- Package installation with pip/conda
- Creating modular code

String Processing

- String methods for data cleaning
- Modern formatting with f-strings
- Regular expressions for pattern matching
- Text extraction from engineering data

Data Science Foundations

- NumPy for numerical computing
- Pandas for data analysis
- Matplotlib for visualization
- SciPy for scientific computing

Practical Applications

- Complete data pipelines
- Engineering data processing
- Quality control automation
- Report generation

Key Programming Concepts Mastered

Core Python Skills

- **Modular Programming:** Organizing code with modules and packages
- **Text Processing:** String manipulation and regex for data extraction
- **Scientific Computing:** NumPy arrays and vectorized operations
- **Data Analysis:** Pandas DataFrames for structured data
- **Visualization:** Creating publication-quality plots

Engineering Problem-Solving Skills

- Process sensor data from multiple sources
- Clean and standardize messy engineering data
- Perform statistical analysis on test results
- Create automated quality control pipelines
- Generate professional technical reports

Questions?

Thank you!

Dr. Eyuphan Koc
eyuphan.koc@bogazici.edu.tr